Engenharia de Software

5º Semestre

Estrutura de Dados II

Implementação, Código e Análise de Análise de Árvores de Busca e Caixeiro – Viajante

Ponta Grossa – PR

Unicesumar

Sumario

1. Introdução 2
2. Implementação e Análise do Código 2
2.1. O motor de Balanceamento da Árvore AVL 2
2.2. A Estratégia de Cores da Árvore Rubro-Negra 3
2.3. a Heurística do Caixeiro-Viajante (TSP) 4
3. Análise do Espaço-Temporal (Big-O) 5
4. Metodologia Experimental 5
4.1. Overhead 6
4.2. Demonstração 6
4.2.1. Menu 6
4.2.2 Árvore BST 6
4.2.3 Árvore AVL 7
4.2.4 Rubro-Negra 8
4.2.5 Caixeiro-Viajante 9
5. Considerações 10

1. Introdução

O projeto teve o objetivo de implementar e analisar três tipos de árvores (Árvore Binária de Busca, Árvore AVL e Árvore Rubro-Negra), além de solucionar um problema clássico de otimização em grafos (Caixeiro Viajante). O foco principal é a análise e o comportamento destras estruturas conforme a inserção dos dados e sua eficiência do balanceamento.

2. Implementação e Análise do Código

Para a análise de complexidade, foi destacado a parte lógica no código do projeto e explicando o funcionamento.

2.1. O motor de Balanceamento da Árvore AVL

A invariante principal da AVL exige que o Fator de balanceamento (FB) obedeça à restrição:

$$FB \in \{-1, 0, 1\}$$

O trecho do código demonstra o cálculo desse fator e o acionamento imediato da correção estrutural (Rotação).

```
private int getFatorBalanceamento(Node n){  1 usage  xLuanKyrb
    if (n == null) return 0;
    return getAltura(n.esquerdo) - getAltura(n.direito);
}
```

```
    if (balanceamento > 1 && valor < node.esquerdo.valor)
        return rotacaoDireita(node);
```

Análise Lógica: O método getFatoBalanceamento age como o “sensor” da árvore. Se o retorno for 2 (Indicando sobrecarga à esquerda) e o novo valor tiver sido inserido à esquerda, o algoritmo aciona a rotacaoDireita() em tempo O(1). Está intervenção garante que a altura global da árvore nunca exceda 1.44 $\log_2 n$.

2.2. A Estratégia de Cores da Árvore Rubro-Negra

A Rubro-Negra substitui a parte rigorosa matematicamente da AVL por propriedades de colocação, sendo a mais crítica: nenhum nó vermelho pode ter filhos vermelhos.

```
while (k.pai.cor == VERMELHO) {
    if (k.pai == k.pai.pai.direito) {
        tio = k.pai.pai.esquerdo;

        if (tio != null && tio.cor == VERMELHO) {
            tio.cor = PRETO;
            k.pai.cor = PRETO;
            k.pai.pai.cor = VERMELHO;
            k = k.pai.pai;
        } else {
            if (k == k.pai.esquerdo) {
                k = k.pai;
                rotacaoDireita(k);
            }
```

Análise, ao invés de rodar a árvore imediatamente, o algoritmo verifica a cor do nó “Tio”. Se for vermelho, a quebra de regra é resolvida apenas alterando booleanos na memória, poupa ciclos de processamento que teriam sido gastos numa rotação complexa. Isso faz ela ser transacionalmente mais barata que a AVL.

2.3. a Heurística do Caixeiro-Viajante (TSP)

Para contornar a complexidade fatorial de O(n!) na busca exata da menor rota, implementa-se a heurística do Vizinho Mais Próximo.

```
for (int j = 0; j < numeroDeCidades; j++) {
    if (!visitadas[j] && grafo[cidadeAtual][j] != 0) {
        if (grafo[cidadeAtual][j] < menorDistancia){
        menorDistancia = grafo[cidadeAtual][j];
        proximaCidade = j;
    }
}
```

Análise, o laço de repetição adota uma estratégia "gulosa": toma a melhor decisão local possível a cada passo da iteração, ignorando o impacto global. Esta escolha reduz drasticamente a complexidade temporal para $O(n^2)$, entregam uma rota altamente eficiente.

3. Análise do Espaço-Temporal (Big-O)

A eficiência teórica das estruturas implementadas é governada pela altura h da árvore:

BST: A complexidade média de busca e inserção é de O(log n). Contudo, no pior caso (dados em sequencia), a topologia degenera para uma lista encadeada, resultando em complexidade linear O(n).
AVL e Rubro-Negra: Os algoritmos na Secção 2, ambas vão limitar rigidamente a sua altura máxima. Portanto, as operações de busca e inserção mantêm uma complexidade estritamente delimitada a O(log n), protegendo o sistema no pior caso.

4. Metodologia Experimental

Para validar a teoria assintótica, estruturou-se um benchmark executando 30 iterações independentes para conjuntos de 100, 1000, 10000 valores aleatórios.

Entradas (N)	Tempo Médio BST	Desvio Padrão BST	Tempo Médio AVL	Desvio Padrão AVL
100	0,2227 ms	1,0144 ms	0,1454 ms	0,1370 ms
1.000	0,2674 ms	0,1633 ms	0,7192 ms	0,4085 ms
10.000	3,0009 ms	0,8412 ms	3,9708 ms	1,1698 ms

4.1. Overhead

Os dados empíricos revelam um fenómeno vital na engenharia de software: para N >= 1000, a BST performou inserções mais rápidas que a AVL.

Matematicamente, ambas operam em O(log n) num cenário aleatório. A diferença é no fator constante de processamento: na BST, o nó é apenas alocado na memória folha. Na AVL, cada inserção desencadeia os recálculos recursivos de altura. Para finalizar a AVL investe tempo de CPU durante a inserção para estabilizar a sua estrutura, de modo a garantir buscas imbatíveis.

4.2. Demonstração

4.2.1. Menu

```
---------------MENU DE ARVORES-------------
1. Árvore Binária de Busca (BST)
2. Árvore AVL
3. Árvore Rubro-Negro
4. Resolver o Caixeiro-Viajante (TSP)
0. Sair do Programa
```

4.2.2 Árvore BST

Altura = 10 (Degeneração estrutural severa)

```
Quantos valores você quer colocar?
10
Digite os 10 valores
1
2
3
4
5
6
7
8
9
10
Valores inseridos com sucesso
\n--- OPERAÇÕES NA ÁRVORE
  1. Inserir múltiplos valores de uma vez
  2. Inserir apenas um valor
  3. Buscar um valor
  4. Remover um valor
  5. Ver Altura da Árvore
  9. Voltar ao Menu Principal
  Escolha a operação:

5
A altura atual da árvore: 10
```

4.2.3 Árvore AVL

Altura = 4 (As rotações matemáticas forçaram a árvore ao formato mais compacto possível)

```
Quantos valores você quer colocar?
10
Digite os 10 valores
1
2
3
4
5
6
7
8
9
10
Valores inseridos com sucesso
\n--- OPERAÇÕES NA ÁRVORE
  1. Inserir múltiplos valores de uma vez
  2. Inserir apenas um valor
  3. Buscar um valor
  4. Remover um valor
  5. Ver Altura da Árvore
  9. Voltar ao Menu Principal
  Escolha a operação:

5
A altura atual da árvore: 4
```

4.2.4 Rubro-Negra

Altura = 5 (O balanceamento de repintura evitou a degeneração, mas permitiu uma pequena inclinação a favor de performance de CPU)

```
Quantos valores você quer colocar?
10
Digite os 10 valores
1
2
3
4
5
6
7
8
9
10
Valores inseridos com sucesso
\n--- OPERAÇÕES NA ÁRVORE
  1. Inserir múltiplos valores de uma vez
  2. Inserir apenas um valor
  3. Buscar um valor
  4. Remover um valor
  5. Ver Altura da Árvore
  9. Voltar ao Menu Principal
  Escolha a operação:

5
A altura atual da árvore: 5
```

4.2.5 Caixeiro-Viajante

O algoritmo heurístico gerou a seguinte saída computacional para o menor trajeto a partir da Cidade 0:

```
Problema do Caixeiro
De qual cidade quer começar? (DIGITE 0 a 3)
0
Resultado do Caixero-Viajante
Caminho:0 ->
1 ->
3 ->
2 ->
0
Custo total (Distância): 80
Iniciando a viagem a partir da Cidade 0
```

5. Considerações

Através da união entre a prova teórica dos códigos e o levantamento estatístico, comprava-se que a escolha da estrutura é um trade-offs (Vantagens e desvantagens). A BST é eficiente para escritas de dados desordenados; a AVL sacrifica desemprenho na inserção para prover o ecossistema de leitura mais eficiente; e a Rubro0Negra posiciona-se como o balanço para sistemas reais. Já o Caixeiro-Viajante ilustrou como deixar mais prático para resolver problemas do mundo real em tempo viável.